FP game design Waizy Giesberts

Masheed Waizy, 0384739
Kevin Giesberts, 7254970

October 2026

1 Introduction and Rules

This document describes our implementation of the game Asteroids. Asteroids is a single-player arcade shooter set in a 2D space that wraps around the screens edges. The player controls a spaceship that can rotate, boost forward, and shoot bullets to break apart moving asteroids (of many sizes) and UFOs. The game ends when the player loses all three lives by hitting an asteroid or an enemy UFO. There is also a scoretracker,

2 Design

2.1 Data structures

2.1.1 The game state

The full state of the game is stored in a GameState record. The current screen is an enumeration, so the menu, game, pause, game over, and high score screens all use one system. We also store the random number generator and high scores in the state so that all game logic can be written as pure functions on GameState.

```
data Screen = Menu | Playing | Paused | GameOver | HighScores
  deriving (Eq, Show)

data GameState = GameState
  { screen      :: Screen
  , ship        :: Ship
  , asteroids   :: [Asteroid]
  , ufos        :: [UFO]
  , bullets     :: [Bullet]
  , explosions  :: [Explosion]
  , score       :: Int
  , lives       :: Int
  , wave        :: Int
  , ufoTimer    :: Float
  , elapsedTime :: Float
```

```
  , rng          :: StdGen
  , highScores   :: [HighScore]
  }
```

The player is always human.

2.1.2 The player and the field

The ship, bullets, and asteroids exist in a flat 2D space. Positions and velocities are 2D vectors. Each entity type has its own data type so the type system keeps them separate (for example, there is always only one Ship).

```
type Vector = (Float, Float)

data TurnDir = TurnLeft | TurnRight | NoTurn
  deriving (Eq, Show)

data Ship = Ship
  { shipPos        :: Vector
  , shipVel        :: Vector
  , heading        :: Float
  , isThrusting    :: Bool
  , turnDir        :: TurnDir
  , shootCooldown  :: Float
  , invulnTime     :: Float
  } deriving (Show)

data AsteroidSize = Large | Medium | Small
  deriving (Eq, Show)

data Asteroid = Asteroid
  { astPos   :: Vector
  , astVel   :: Vector
  , astSize  :: AsteroidSize
  } deriving (Show)

data BulletOwner = PlayerBullet | UFOBullet
  deriving (Eq, Show)

data Bullet = Bullet
  { bulletPos      :: Vector
  , bulletVel      :: Vector
  , bulletLifeTime :: Float
  , bulletOwner    :: BulletOwner
  } deriving (Show)

data Explosion = Explosion
  { expPos      :: Vector
  , expScale    :: Float
  , expTimeLeft :: Float
```

```
  } deriving (Show)

data HighScore = HighScore
  { hsTimestamp :: Integer
  , hsScore     :: Int
  } deriving (Show, Generic)
```

The UFO type is explained in Section 2.2. The playing field uses a continuous coordinate system that wraps around the screens edges. All number settings (screen size, turn speed, thrust, drag, bullet speed and lifetime, points per asteroid, and high score file name) are stored as named constants in a seperate file to avoid contamination.

2.1.3 Movement

The player controls the ship with the keyboard: Up to thrust, left and right to turn, space to shoot, and p to pause. Because Gloss sends key-down and key-up events, inputPure only updates the isThrusting and turnDir flags on the ship. Then, stepPure applies these flags every frame using the elapsed time. This keeps input handling and physics separate. The ship keeps moving after the player releases the thrust key, and drag slowly slows it down.

```
applyTurn   :: Float -> Ship -> Ship
applyThrust :: Float -> Ship -> Ship
applyDrag   :: Float -> Ship -> Ship
fireBullet  :: Ship -> (Ship, Maybe Bullet)
wrap        :: Vector -> Vector
```

Screen wrap-around is written once in wrap, which is used by the generic move function from the Movable typeclass (Section 6). Any object that leaves one side of the screen appears on the opposite side with the same velocity, so the ship keeps its speed and direction. Only bullets wrap the same way and they disappear when their lifetime runs out.

2.2 Hostile entities

The game has two types of enemies that behave differently: Asteroids and UFOs.

2.2.1 Asteroids

Asteroids move at a constant speed (depending on their size). When hit by enough player bullets, an asteroid splits into two smaller asteroids moving in random directions. A small asteroid is simply destroyed. Because splitting and spawning use random numbers, we pass the random generator directly:

```
splitAsteroid :: StdGen -> Asteroid -> ([Asteroid], StdGen)
spawnWave     :: Int -> StdGen -> ([Asteroid], StdGen)
```

spawnWave creates Large asteroids at random positions along the screen edges with random velocities. The number of asteroids increases with each wave.

2.2.2 UFOs

UFOs have basic intelligence. They move toward a target position near the player (updated regularly) and fire a UFOBullet at the player when shootTimer reaches zero. A big UFO shoots with a small random angular error, while a small UFO aims accurately and only appears starting at later waves.

```
data UFOKind = BigUFO | SmallUFO
  deriving (Eq, Show)

data UFO = UFO
  { ufoKind    :: UFOKind
  , ufoPos     :: Vector
  , ufoVel     :: Vector
  , targetPos  :: Vector
  , shootTimer :: Float
  } deriving (Show)

updateUFO   :: Float -> Ship -> StdGen -> UFO -> (UFO, Maybe Bullet, StdGen)
aimAtPlayer :: Ship -> UFO -> Vector
spawnUFO    :: Int -> StdGen -> (UFO, StdGen)
```

2.2.3 Collisions

All objects have circular hitboxes, so a single generic collides function handles all checks. The collision rules are:

- Player bullet and asteroid: the asteroid splits, the player gets points, and an explosion appears.
- Player bullet and UFO: the UFO is destroyed and the player gets points.
- Ship and asteroid, UFO, or UFO bullet: the player loses a life, an explosion appears, and the ship respawns with temporary invulnerability (collisions are ignored while invulnerablity timer ¿= 0).

```
resolveCollisions :: GameState -> GameState
```

3 Implementation of the Minimum Requirements

3.1 Player

The player controls the spaceship with the arrow keys.

Player: The player controls one character using the keyboard.

Enemies: The game features enemy characters with that try to kill the lpayer. Enemies update their positions each frame in stepPure using their velocity vectors and targeting logica.

3.2 Randomness

Random numbers are used for: asteroid positions, sizes, and speeds at the start of a wave; the directions of split asteroid pieces; the screen edge where a UFO spawns; the time between UFO spawns; and the aiming error of big UFOs. Randomness stays pure because the generator is kept in the rng field of GameState, and every function that needs randomness takes a StdGen and returns an updated one.

```
randomIn      :: (Float, Float) -> StdGen -> (Float, StdGen)
randomEdgePos :: StdGen -> (Vector, StdGen)
spawnWave     :: Int -> StdGen -> ([Asteroid], StdGen)
spawnUFO      :: Int -> StdGen -> (UFO, StdGen)
```

stepPure reads rng from the state, passes it through the spawn and split functions, and saves the new generator back into the state. The initial generator is created in main using getStdGen. Because of this design, the game always behaves the same way for a given starting condition, which makes it easy to test.

3.3 Animation

Animations are driven by timers in the game state, keeping viewPure a pure function of GameState:

- **Explosion:** Created whenever an object is destroyed. stepPure lowers expTimeLeft by dt and removes the explosion when it reaches zero. The view calculates progress as p = 1 - time left / explosion duration and draws a growing, fading ring with radius expScale * p.
- **Thrust flame:** Drawn behind the ship while isThrusting is True.
- **Respawn blinking:** While invulnTime > 0, the ship is only drawn on alternating time intervals of elapsedTime, making it blink.
- **Screen transitions:** Switching between Menu, Playing, Paused, GameOver, and HighScores is done by updating the screen field.

```
instance Renderable Explosion where
  render :: Explosion -> Picture
thrustFlame :: Float -> Picture
```

3.4 Pause

The player can pause the game by pressing the p button. inputPure listens for this key and switches the screen between Playing and Paused. In stepPure, if the screen is Paused, all game updates are skipped, which freezes all movement and timers.

3.5 Interaction with the File System

Game scores are saved in a JSON file on the player's computer. The high score system uses file IO functions.

```
loadHighScores :: FilePath -> IO [(String, Int)]
saveHighScore  :: FilePath -> String -> Int -> IO ()
```

When a game ends, saveHighScore saves the Unix timestamp and score to the file and updates highScores in GameState. On the high score screen, the player can view all past scores and when they were achieved.

4 Implementation of the optional Requirements

We implemented one optional requierement.

4.1 Different enemies

The game has three enemy types that act differently:

- **Asteroids** drift passively and split into smaller pieces when shot.
- **Big UFOs** follow the player and shoot with random aiming error.
- **Small UFOs** aim accurately and shoot faster.

Harder enemies show up in later waves: spawnUFO only spawns a SmallUFO from wave 6 onward, and ufoTimer gets shorter as the wave number increases. This way the difficulty keeps increasing at a steady pace.

5 Interface and separation of pure and impure code

We use the gloss library with playIO, The game only uses impure code for three things: reading and writing the high score file, generating a random number, and closing / starting the game. Everything else should be pure code:

```
viewPure  :: GameState -> Picture
inputPure :: GameState -> GameState
stepPure  :: Float -> GameState -> GameState

initial :: StdGen -> [HighScore] -> GameState
view    :: GameState -> IO Picture
input   :: Event -> GameState -> IO GameState
step    :: Float -> GameState -> IO GameState
```

In main, we get the random generator with getStdGen, load the high scores with loadHighScores, and pass both to initial. The view and input functions

simply wrap the pure functions using return . viewPure and return . inputPure e.

The score only needs to be saved once per game, not every frame. step compares the screen state before and after calling stepPure, and only saves the score when the screen changes from playing to GameOver.

6 Abstraction

Ship, Asteroid, UFO, and Bullet all move in the same 2D space, wrap around the screen, and use circular hitboxes. We use typeclasses to avoid repeating code. Each data type only defines simple helper functions, while move and collides are written once for all of them. This allows us to move any list of objects remove expired bullets and explosions with updateTimed, check collisions using higher-order functions like any and partition, and draw objects with map render.